

EC 504
Fall, 2025
Homework 4

Due Tuesday, November 4, 11:59 pm on your SCC account

Problem 4.1 (20 pts)

Below are a set of quick true false type questions on graphs. Please provide brief answers.

- (a) An undirected graph with no cycles is a tree.

Solution:

False, as there is no guarantee the graph is connected.

- (b) Consider an undirected graph where, for every pair of nodes i, j , there exists a unique simple path between them. This graph must be a tree.

Solution:

True. The graph cannot have cycles, and it must be connected, hence it is a tree.

- (c) Kruskal's algorithm for finding a minimum spanning tree grows a single tree into a minimum spanning tree.

Solution:

False, Kruskal's algorithm grows a forest.

- (d) The worst case complexity of the fastest minimum spanning tree algorithm is $O(n \log n)$, where n is the number of vertices in the graph.

Solution:

False, the fastest MST algorithm is $O(m + n \log(n))$, where m is the number of edges.

- (e) Consider a minimum spanning tree T in a connected undirected graph (V, E) . Assume that the edge weights in the graph are all distinct (e.g. no two edges have the same weight). Then, an edge i, j in T must have the property that it is either the minimum weight edge connected to vertex i or the minimum weight edge connected to vertex j .

Solution:

False. Consider a simple graph with four nodes arranged in a line, where the weight from 1 to 2 is 1, the weight from 2 to 3 is 3, and the weight from 3 to 4 is 2. This graph is already a minimum spanning tree. Note that the edge from 2 to 3 is in the MST, but is neither the smallest weight edge connected to 2 or the smallest weight edge connected to 3.

- (f) Vertices v_1 and v_2 are neighbors in an undirected graph if and only if there is a path between them.

Solution:

False, there must be an edge between them.

- (g) Prim's algorithm and Kruskal's algorithm can also be used to find the maximum weight spanning tree in a graph.

Solution:

True. To find the maximum weight, simply compute the negative value of the weights and find the minimum weight. Note that you can always add a constant to every arc, and get the same minimum spanning tree, since the number of arcs is the same for every spanning tree. Thus, you can solve the problem with negative weight arcs.

- (h) Consider a minimum spanning tree in an undirected graph $G = (V, E)$ where all the edge weights are distinct. Then, if an edge e has the largest weight of all the edges in a cycle, that edge e cannot be part of a minimum spanning tree.

Solution:

True. If e is in a spanning tree T , then there is an edge e' in the cycle that is not in T (because a tree can have no cycles), and has weight less than e . Then, replacing e with e' results in a new spanning tree that has less weight than T , hence T cannot be a minimum spanning tree.

- (i) If a weighted, undirected graph has unique weights on its edges, the smallest weight edge on every cycle must be part of the minimum spanning tree.

Solution:

False. Consider a fully connected graph with four vertices, and thus 6 edges. Assign edge weights $w_{12} = 1, w_{13} = 2, w_{14} = 3, w_{23} = 4, w_{24} = 5, w_{34} = 6$. Then, the MST has the edges $w_{12} = 1, w_{13} = 2, w_{14} = 3$, and none of the edges in the cycle 2-3-4 are in the MST.

- (j) If you add an extra edge to a minimum spanning tree, you form a graph with exactly one cycle.

Solution:

True. If the edge is $\{i, j\}$ and is not in the MST, then the one cycle formed has the edges in the MST in the path connecting i and j , plus the edge $\{i, j\}$.

Problem 4.2 (20 pts)

Consider an undirected graph $G = (V, E)$.

- (a) Describe an $O(|V|)$ -time algorithm for determining whether the graph contains a cycle. .

Solution:

Do a Depth-First-Search algorithm on the graph. If, while doing DFS, you find an edge that points to a node already in the stack for DFS which is not the predecessor node that put it in the stack, this is an edge that creates a cycle.

This is $O(|V|)$ because, if the graph has no cycles, it only has at most $|V| - 1$ edges, so DFS is $O(|V|)$. If there is a cycle, a back edge will be found no later than when DFS reaches the end of a connected component, which is $O(|V|)$.

You can also do it with breadth-first search, keeping track of predecessor nodes for edges so that we don't count as a cycle an edge connecting the same two nodes. Whenever you find an edge connecting a node at a given depth to another node at the same depth or lesser edge where the second node is not the predecessor node, this creates a cycle.

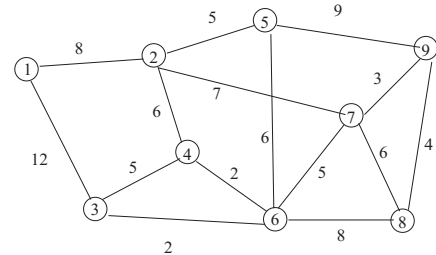
- (b) Describe an algorithm to find the minimum number of edges that must be removed from a given undirected graph in order to make it acyclic. Note the graph may not be connected.

Solution:

Note that we don't have to find the edges to remove, just the minimum number. If the graph is connected, this is a simple algorithm: remove $|E| - |V| + 1$ edges. If the graph has k connected components, then you must remove a minimum of $|E| - |V| + k$ edges. Hence, do a DFS or BFS algorithm to find the number of connected components, and provide the answer. Complexity of this is $O(|E|)$.

Problem 4.3 (10 pts)

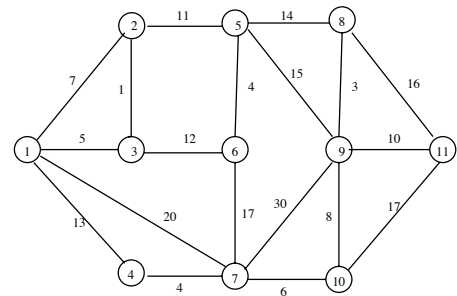
Consider the undirected, weighted graph on the right. List the order in which the nodes will be visited in a Breadth-First Search, starting from node 5. Assume that the arcs out of a node will be visited in clockwise order, starting from straight up direction. Show the tree which is generated by the relation that node i 's parent is the node which puts node i into the queue.

**Solution:**

Starting from node 5, we have, into the queue: Add node 5 to queue and mark it as visited. Remove 5, and add its neighbor nodes 9, 6, 2 to queue and mark as visited. Remove 9, and add 8, 7, mark as visited. Remove 6, and add 3, 4, and mark as visited. Remove 2, and add unvisited neighbor 1 to the queue. Remove nodes in order from queue, to get order 5 - 9 - 6 - 2 - 8 - 7 - 3 - 4 - 1. The tree is

**Problem 4.4** (10 pts)

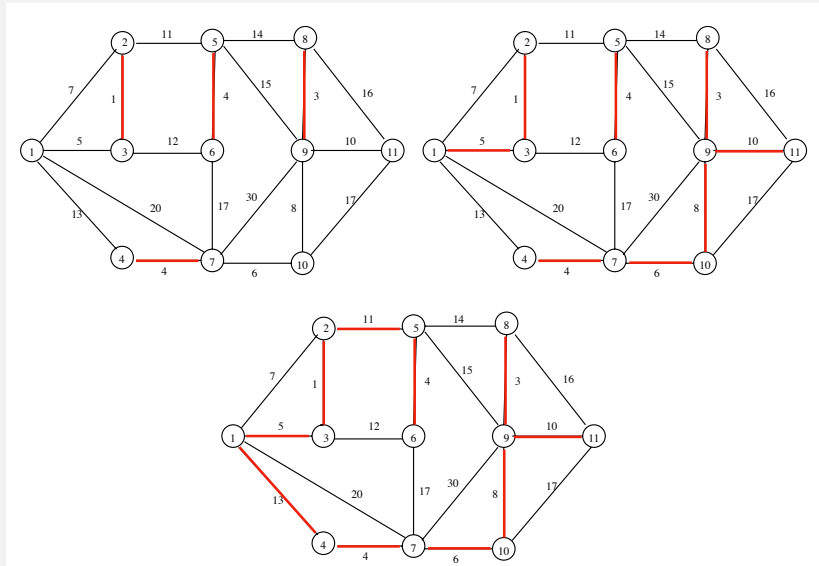
For the graph on the right, compute the minimum spanning tree using Kruskal's algorithm. Sort the edge weights, and show the partial forest of trees that occur in Kruskal's algorithm after **inserting** every 4 edges. Compute the total weight of the minimum spanning tree.

**Solution:**

Sort the edges by increasing weight:

$\{2, 3\}$:	1
$\{8, 9\}$:	3
$\{4, 7\}$:	4
$\{5, 6\}$:	4
$\{1, 3\}$:	5
$\{7, 10\}$:	6
$\{1, 2\}$:	7
$\{9, 10\}$:	8
$\{9, 11\}$:	10
$\{2, 5\}$:	11
$\{3, 6\}$:	12
$\{1, 4\}$:	13
$\{5, 8\}$:	14
$\{5, 9\}$:	15
$\{10, 11\}$:	17
$\{1, 7\}$:	20
$\{7, 9\}$:	30

Total weight: 65



Problem 4.5 (20 pts)

Consider a path between two vertices s and t in an undirected, *connected* weighted graph G where no two edges have the same weight. The bottleneck length of this path is the maximum weight of any edge in the path. The bottleneck distance between s and t is the minimum bottleneck length of any path from s to t . Describe an algorithm to compute the bottleneck distance between every pair of vertices in an arbitrary undirected weighted graph. (Hint: Minimum spanning trees will help...)

Solution:

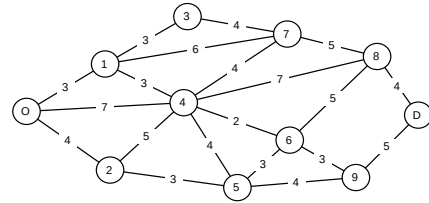
Find a MST (V, T) . The bottleneck distance between any two vertices will be the maximum edge weight in the unique path between two vertices in the MST.

Why is this true? Assume it is not: Let d^* be the bottleneck distance between s and t in (V, E) , and let $d > d^*$ be the bottleneck distance between s and t in the MST (V, T) . Then, there is an edge e in the path p from s to t in (V, T) such that $w(e) = d > d^*$. There is also a path p_1 from s to t in (V, E) where the weight of all edges is less than or equal to d^* . Thus, if one were to execute Kruskal's algorithm, vertices s, t would be in the same connected component once all edges of weight less than or equal to d^* were considered. Thus, the edge e with weight $d > d^*$ cannot be part of an MST.

The complexity for finding an MST is $O(|E| + |V| \log(|V|))$. Now, how can we find the bottleneck distance for every pair of vertices from the MST? If you think about this, you are generating $O(|V|^2)$ outputs, so any solution is at least $\Omega(|V|^2)$. There are many ways of doing this. One easy way is to pick the largest weight edge in the MST, and find the two connected components in the rest of the MST. Then, the bottleneck distance between every pair of vertices, one in one component, one in the other is the largest weight. We now have two smaller equivalent problems, with one less edge. The worst case is when only one vertex is in one connected component, so we reduce the size of the problem by 1. We have to do this iteration $|V|$ times, and the complexity of each iteration is $O(|V|)$, so we have an $O(|V|^2)$ algorithm, which was the best we could hope for.

Problem 4.6 (10 pts)

Consider the undirected weighted graph in the right. Show each step of Prim's algorithm for finding a minimum spanning tree in the graph.



Solution:

